

PRACTICAL EXAM

Field	Value
Subject Code	SWE202c
Subject Name	Introduction to Software Engineering
Page No	1
Topic	University Library Management System
Time	120 minutes
Tools	MS Word, Draw.io, MS Visio, Astah, Visual Paradigm, StarUML
Note	Open book. Students are NOT allowed to use the Internet.

I. CASE STUDY

VietTech University is a large multi-disciplinary public university in Vietnam, headquartered in Hanoi with two additional campuses in Da Nang and Can Tho. The university serves approximately 45,000 enrolled students, 1,800 lecturers and administrative staff, and operates one central library plus three branch libraries that collectively hold around 520,000 physical volumes and 85,000 ebooks/digital resources licensed from international publishers. For the past decade, each library branch has been running its own legacy desktop catalogue together with ad-hoc Microsoft Excel spreadsheets to track loans, fines, and inventory. Because the four branches do not share a unified database, the same physical book can appear "available" in two branches simultaneously, lost copies are discovered only during yearly stocktake, and inter-branch lending is essentially impossible. At the start of the 2024–2025 academic year, a faulty overnight Excel sync corrupted the loan ledger and left more than 3,200 freshmen unable to borrow the prescribed coursebooks during the first week of class, triggering formal complaints to the Ministry of Education. The university board has therefore approved a strategic project to build a purpose-built **University Library Management System (ULMS)**, internally code-named **SmartLib**, to be in production before the start of the Spring 2026 semester.

The system must serve five distinct user groups. Members (students and faculty) need to search the unified catalogue using filters such as title, author, ISBN, subject category, language, publication year, branch availability, and rating, view rich book detail pages with cover image, table of contents, and reader reviews, place holds/reservations on currently borrowed copies, perform self-checkout at RFID kiosks, renew or return loans online, read licensed ebooks through an in-browser DRM viewer, and pay overdue fines via e-wallet or bank transfer. Librarians must catalogue new acquisitions (capturing MARC21 metadata), tag and bind RFID

labels to each physical copy, process counter checkouts and returns, mediate damage assessments, manage hold queues, and issue fine waivers within their authorised limit. Library Managers oversee one or more branches and must configure opening hours, loan-period rules per member category, run inventory stocktake campaigns, approve large fine waivers, view branch-level analytics dashboards (circulation volume, top-borrowed titles, late-return ratio, ebook usage), and submit acquisition budget proposals. Customer Support Librarians handle complex member complaints, lost-item replacements, and inter-branch transfer disputes. Finally, the System Administrator is responsible for user-and-role management, RFID-reader and payment-gateway configuration, ebook-DRM key rotation, automated backups, and platform-wide monitoring. The system must additionally allow External Readers (alumni, visiting researchers, partner-institution staff) to register a paid temporary membership and use a restricted subset of services.

The system must satisfy several demanding technical constraints. At the start of every semester the catalogue search and reservation pages may be hit by up to 8,000 concurrent users within minutes, so all critical pages (search, book detail, member dashboard, and checkout) must load within 2 seconds under peak load, and the loan-transaction pipeline must sustain at least 400 transactions per minute without producing duplicate or phantom loans. Each RFID self-checkout scan must be acknowledged on screen within 1 second, and any change in copy availability must propagate to all four branches in under 5 seconds. All member personal data, membership-card credentials, fine-payment records, and ebook-DRM keys must be encrypted in transit (TLS 1.3) and at rest (AES-256), and the system must comply with Vietnam's Personal Data Protection Decree (PDPD 2023). Access must be enforced through a role-based access control (RBAC) model covering all five user categories. The platform must target 99.9% monthly availability, with planned maintenance restricted to the 02:00–03:30 local-time window. The mobile-first member portal must render correctly on screens as narrow as 360 px, support both Vietnamese and English, and pass WCAG 2.1 Level AA standards. The architecture must support horizontal auto-scaling and be geo-distributed across the three campus regions (Hanoi, Da Nang, Can Tho) so that additional application nodes can be added transparently behind regional load balancers as the catalogue and member base continue to grow.

II. QUESTIONS

Answer ALL questions. Base your answers on the **University Library Management System (ULMS / SmartLib)** case study above. Where diagrams are required, use standard UML notation and label all elements clearly.

Question 1: Software Development Model (2.0 points)

1. Identify and justify the most appropriate software development model (SDLC) for the SmartLib project. In your justification, address each of the following points: (1.0 points)
 - Whether all requirements are fully known and stable at project inception

- The need for iterative delivery, regular stakeholder feedback, and early risk mitigation
 - Risk factors specific to SmartLib (e.g., legacy data migration from Excel, RFID hardware integration, ebook DRM licensing, multi-branch real-time synchronisation, semester-start traffic spikes)
 - Team size, projected project duration, and the university's capacity to accommodate evolving requirements as cataloguing rules and inter-branch lending policies are refined
2. Select ONE alternative SDLC model and compare it with your chosen model in the context of SmartLib. Your comparison must include at least two advantages and two disadvantages for each model. (1.0 points)

Question 2: UC Modeling (1.5 points)

1. Identify FIVE actors in the University Library Management System. For each actor, state their primary goal and whether they are a primary or secondary actor. (0.3 points)
2. List at least 6 use cases for SmartLib. For each use case, state the initiating actor(s) and a one-sentence description of the goal. (0.4 points)
3. Draw a complete UML Use Case Diagram for SmartLib. The diagram must include: (0.8 points)
 - All identified actors placed correctly inside or outside the system boundary
 - All major use cases inside the system boundary rectangle
 - At least ONE <<include>> relationship — provide a brief written rationale
 - At least ONE <<extend>> relationship — provide a brief written rationale
 - Any actor-generalisation relationships that apply

Question 3: UC Specification (1.5 points)

Write a complete Use Case Specification for the use case '**Borrow Book**' (a member checks out a physical book at an RFID self-checkout kiosk, from card scan through to receipt printing). Use the template in the answer sheet and fill in EVERY field. Partial or missing sections will receive partial credit only.

Question 4: Non-Functional Requirements (1.0 points)

1. From the case study, identify and classify at least FOUR non-functional requirements (NFRs) into appropriate quality categories (e.g., Performance, Security, Availability, Usability, Scalability, Maintainability). For each NFR, write one measurable acceptance criterion. Present your answer as a table with the columns: **Category | NFR Description | Acceptance Criterion**. (0.6 points)
2. Identify TWO NFRs from your table that may conflict with each other in the SmartLib context. Clearly explain why the conflict arises and propose a concrete trade-off strategy that the development team could apply. (0.4 points)

Question 5: Class Diagram (1.5 points)

1. Identify at least SIX key domain classes for SmartLib. For each class, specify at minimum three attributes (with data types) and two methods. Present this as a structured list or mini class-description table before drawing the diagram. (0.6 points)
2. Draw a complete UML Class Diagram that includes: (0.9 points)
 - All identified classes with their full attribute and method compartments
 - At least ONE inheritance (generalisation) relationship — for example, User as a base class for Member, Librarian, LibraryManager, and SystemAdmin
 - At least TWO association relationships with correct multiplicity labels at both ends
 - At least ONE aggregation OR composition relationship, with correct diamond notation and a written justification
 - At least ONE dependency relationship

Suggested classes (you may add or rename): User, Member, Librarian, LibraryManager, SupportLibrarian, SystemAdmin, Branch, Book, BookCopy, Author, Category, Loan, Reservation, Fine, Payment, MembershipCard, Ebook, EbookLicense, Review, RFIDTag, AcquisitionRequest.

Question 6: AI in coding (1.0 points)

This question assesses your ability to use AI coding assistants (e.g., GitHub Copilot, ChatGPT, Claude) as a practical aid during software development. Two scenarios are provided below. For each scenario, you must craft an effective AI prompt and then answer the follow-up tasks.

Important: A good AI prompt must include: (1) clear context about the system, (2) the specific goal, (3) the code or project information itself, and (4) the expected output format.

Scenario — Structure Project Source Code with MVC Architecture

The SmartLib back-end is being built with Java Spring Boot. The development team has started the project but all classes (controllers, services, repositories, models, DTOs, and configuration files) are placed in a single package `com.viettech.smartlib` without any further organisation. The team wants to refactor the project into a clean MVC (Model-View-Controller) architecture so that the codebase is easier to maintain, test, and scale. The current flat package contains the following classes:

```
com.viettech.smartlib
├─ BookController.java           // handles HTTP requests for catalogue
search and book details
├─ LoanController.java          // handles HTTP requests for
borrow/return/renew operations
├─ MemberController.java        // handles HTTP requests for member
```

```

registration and profile
├─ ReservationController.java      // handles HTTP requests for placing and
cancelling reservations
├─ BookService.java                // business logic for catalogue and
inventory
├─ LoanService.java                // business logic for loan lifecycle and
fine calculation
├─ ReservationService.java         // business logic for hold queues
├─ FineService.java                // business logic for overdue fines and
payments
├─ BookRepository.java             // JPA repository for Book entity
├─ LoanRepository.java             // JPA repository for Loan entity
├─ MemberRepository.java           // JPA repository for Member entity
├─ ReservationRepository.java      // JPA repository for Reservation entity
├─ Book.java                       // JPA entity
├─ Loan.java                       // JPA entity
├─ Member.java                     // JPA entity
├─ Reservation.java                // JPA entity
├─ LoanDTO.java                    // data transfer object
├─ ReservationRequestDTO.java      // data transfer object
├─ JwtAuthFilter.java              // Spring Security filter
└─ AppConfig.java                  // application-level configuration

```

Your tasks for Scenario:

- **Write an AI Prompt to Restructure the Project.** Write a clear and complete prompt you would submit to an AI assistant to reorganize the flat package above into a proper MVC directory structure. Your prompt must specify: (1) the system context (SmartLib, Java Spring Boot), (2) the goal (restructure into MVC with clear package naming), (3) the full list of existing classes provided above, and (4) the expected output format (a proposed directory tree with package names and brief justification for each layer). (0.4 points)
- **Identify the MVC Layers.** Based on the AI's proposed structure, map each class to its correct MVC layer (Model, View/DTO, Controller, Service, Repository, Config/Security). Present this as a table with columns: **Class Name | MVC Layer | Justification**. (0.4 points)
- **Write a Follow-up AI Prompt.** Write a follow-up prompt asking the AI to generate the refactored package declaration and import statements for `LoanController.java` and `FineService.java`, reflecting the new MVC package structure. (0.2 point)

Question 7: Testing Stage and Type (1.5 points)

1. Map the four standard testing stages to the SmartLib development process. For each stage, state what is tested, who is responsible, and which testing types are most applicable: (0.5 points)

2. Write THREE test cases for the '**Process Book Return with Late Fine**' use case (a member returns a previously borrowed physical book at the counter or a self-return chute, and the system must compute any overdue fine and update inventory), covering the scenarios listed below. Present each test case in a table with the columns: **Test Case ID | Test Case Name | Precondition | Test Steps | Expected Result | Testing Type**. (1.0 points)
- **Happy-path scenario** — the book is returned before the due date in good condition; no fine is generated and the copy is marked available
 - **Boundary/edge-case scenario** — the book is returned exactly on the due date at 23:59 local time, on the boundary between on-time and overdue
 - **Negative/error scenario** — the book is returned five days late and shows minor damage, so the system must compound a daily overdue fine with a damage-assessment fine and notify the member

III. SCORING RUBRIC

Q#	Topic	Max Score	Weight	Grader's Score
1	Software Development Model Selection	2.0	20%	
2	UC Modeling	1.5	15%	
3	UC Specification	1.5	15%	
4	Non-Functional Requirements	1.0	10%	
5	Class Diagram	1.5	15%	
6	AI in Coding	1.0	10%	
7	Testing Stage and Type	1.5	15%	
	TOTAL	10.0	100%	

IV. SUBMISSION GUIDELINES

4.1 Submission Format

- The question answers must be in the provided template.
- All work must be submitted as a single .zip file.
- File naming: StudentID_SWE202c.zip (e.g., SE123456_SWE202c.zip)
- The .zip must contain: Answer document in MS Word .docx format.

4.2 Formatting Requirements

- Insert answers below each question. Each question should begin on a new page where possible.
- Clearly label the question number and sub-question before each answer section.
- UML diagrams must be clear, titled, and use correct standard notation.
- Font: Times New Roman or Arial, size 12–13 pt, line spacing 1.5.

4.3 General Evaluation Criteria

- Content accuracy and completeness as required by each question.
 - Correct UML syntax, clear and well-annotated diagrams.
 - Logical reasoning tied to the specific context of SmartLib.
 - Clean, professional presentation.
-

--- END OF EXAM ---

Label all diagrams clearly and justify your design decisions wherever asked.